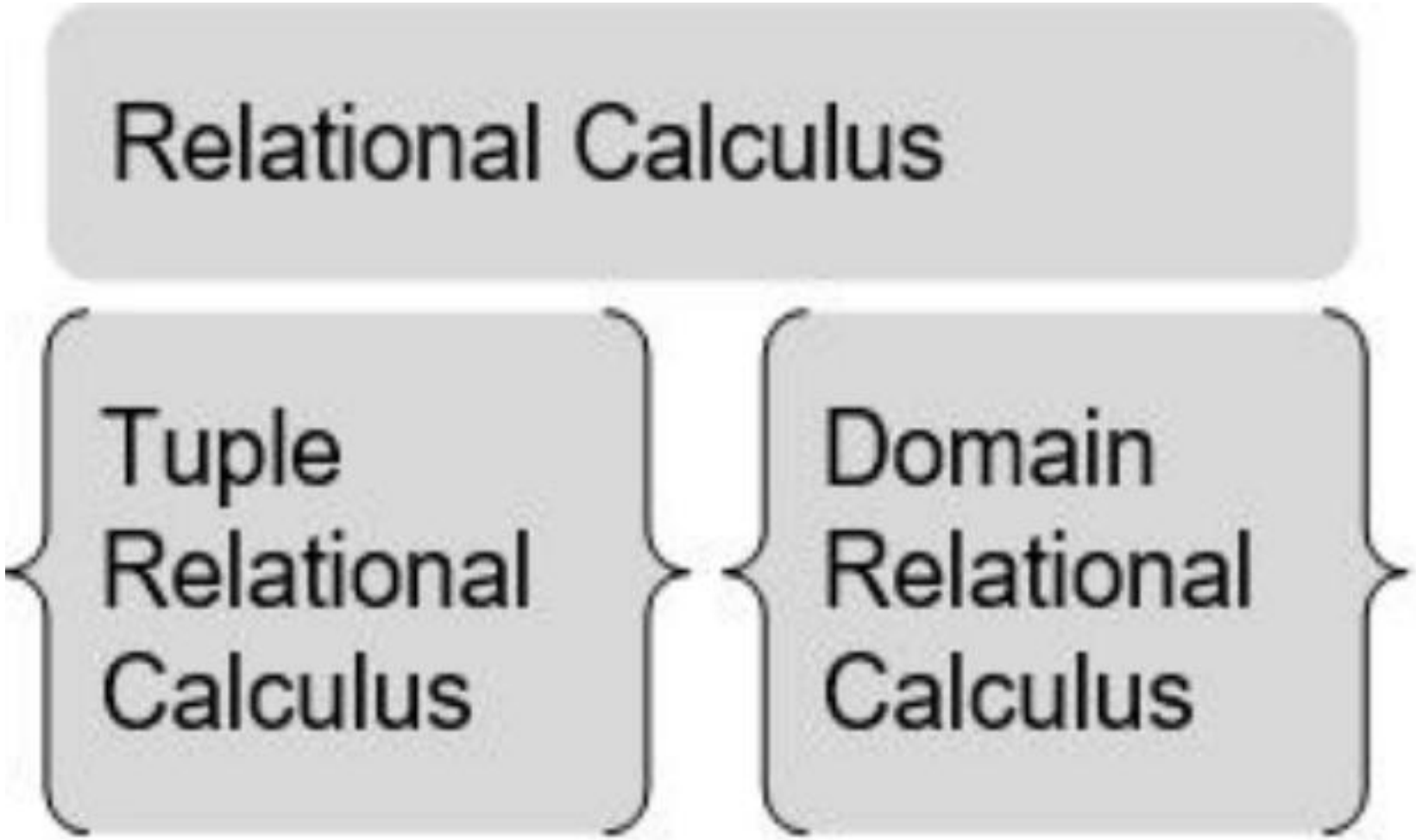
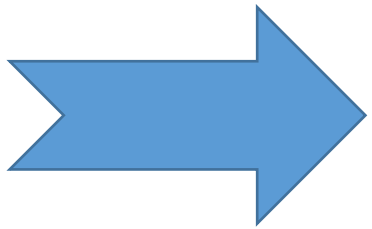
Relational Calculus
DBMS

Relational calculus is a non-procedural query language that tells the system what data to be retrieved but doesn't tell how to retrieve it.

Instead of algebra, it uses mathematical predicate calculus. The relational calculus is not the same as that of differential and integral calculus in mathematics but takes its name from a branch of symbolic logic termed as predicate calculus.


Relational Calculus
Tuple Relational Calculus
Domain Relational Calculus

Tuple Relational Calculus (TRC)



- **Tuple relational calculus is used for selecting those tuples that satisfy the given condition.**

- **The calculus is dependent on the use of tuple variables. A tuple variable is a variable that 'ranges over' a named relation: i.e., a variable whose only permitted values are tuples of the relation.**

Notation:

```
{T | P (T)}  or {T | Condition (T)}
```

Where

T is the resulting tuples

P(T) is the condition used to fetch T.

For example:

```
{ T.name | Author(T) AND T.article = 'database' }
```

OUTPUT: This query selects the tuples from the AUTHOR relation. It returns a tuple with 'name' from Author who has written an article on 'database'.

TRC (tuple relation calculus) can be quantified. In TRC, we can use Existential (∃) and Universal Quantifiers (∀).

For example:

```
{ R| ∃T ∈ Authors(T.article='database' AND R.name=T.name)}
```

Output: This query will yield the same result as the previous one.

Let Rel $\rightarrow$ be a relation name,

R, S $\rightarrow$ be the tuple variables

a $\rightarrow$ an attribute of R

b $\rightarrow$ an attribute of S

op $\rightarrow$ operator in the set $\{<, \leq, >, \geq, =, \neq\}$

An Atomic formula is one of the following:

- R $\in$ Rel
- R.a op S.b
- R.a op Constant or Constant op R.a

To represent the join and division of relational algebra by relational calculus, we need quantifiers such as: ***existential for join*** *and* ***universal for division***

A quantifier quantifies or indicates the quantity of something

The existential quantifier ($\exists$) states that at least one instance of a particular type of thing exist
Similarly, the universal quantifier($\forall$) states that some condition applies to all or to every row of some type

A formula is recursively defined by using the following rules:

- Any atomic formula
- If p and q are formulae, then $\neg$ p, p $\wedge$ q, p $\vee$ q, or p $\Rightarrow$ q are also formulae
- If p is a formula that contains T as a variable, then $\exists$ T(p) and $\forall$ T(p) are also formulae

The quantifiers $\exists$ and $\forall$ are said to **bind** the tuple variable R; whereas a variable is said to be **free** in a formula if the formula does not contain an occurrence of a quantifier that binds it

In most of the queries, the output is shown by using the free variables

Table: Student

First_Name	Last_Name	Age
Ajeet	Singh	30
Chaitanya	Singh	31
Rajeev	Bhatia	27
Carl	Pratap	28

Query to display the last name of those students where age is greater than 30

```
{ t.Last_Name | Student(t) AND t.age > 30 }
```

Output:

Last_Name
Singh

Query to display all the details of students where Last name is 'Singh'

```
{ t | Student(t) AND t.Last_Name = 'Singh' }
```

Output:

First_Name	Last_Name	Age
Ajeet	Singh	30
Chaitanya	Singh	31

Sailor Database

Sailors(sid, sname, rating, age)

Boats(bid, bname, color)

Reserves(sid, bid, day)

Query: Find the names & ages of sailors with a rating above 4

$\{T/\exists S \in \text{Sailors}\ (S.\text{rating} > 4 \wedge T.\text{sname}=S.\text{sname} \wedge T.\text{age}=S.\text{age})\}$

Query: Find the sailor name, boat id & reservation date for each reservation

$\{T/\exists R \in \text{Reserves}\ \exists S \in \text{Sailors}\ (R.\text{sid} = S.\text{sid} \wedge T.\text{sname}=S.\text{sname} \wedge T.\text{bid}=R.\text{bid} \wedge T.\text{day}=R.\text{day})\}$

Query: Find the names of sailors who have reserved boat 111

$\{T/\exists R \in \text{Reserves}\ \exists S \in \text{Sailors}\ (R.\text{sid} = S.\text{sid} \wedge R.\text{bid}=111 \wedge T.\text{sname}=S.\text{sname})\}$

Sailor Database

Sailors(sid, sname, rating, age)

Boats(bid, bname, color)

Reserves(sid, bid, day)

Query: Find the names of sailors who have reserved a green boat

$\{T/\exists S \in Sailors\ \exists R \in Reserves(R.sid = S.sid \wedge T.sname=S.sname \wedge \exists B \in Boats\ (B.bid=R.bid \wedge B.color='green'))\}$

This query can also be written as:

$\{T/\exists S \in Sailors\ \exists R \in Reserves\ \exists B \in Boats(R.sid = S.sid \wedge B.bid=R.bid \wedge B.color='green' \wedge T.sname=S.sname)\}$

Query: Find the names of sailors who have reserved at least 2 boats

$\{T/\exists S \in Sailors\ \exists R1 \in Reserves\ \exists R2 \in Reserves(S.sid=R1.sid \wedge R1.sid= R2.sid \wedge R1.bid \neq R2.bid \wedge T.sname=S.sname)\}$

Sailor Database

Sailors(sid, sname, rating, age)

Boats(bid, bname, color)

Reserves(sid, bid, day)

Query: Find the names of sailors who have reserved all boats

$\{T/\exists S \in Sailors\ \forall B \in Boats(\exists R \in Reserves\ (S.sid=R.sid \wedge R.bid= B.bid \wedge T.sname= S.sname))\}$

Query: Find sailors who have reserved all green boats

$\{S/S \in Sailors \wedge \forall B \in Boats\ (B.color='green' \Rightarrow(\exists\ R \in Reserves\ (S.sid=R.sid \wedge R.bid= B.bid))\}$

Customer (Customer name, Street, City)

Branch (Branch name, Branch city)

Account (Account number, Branch name, Balance)

Loan (Loan number, Branch name, Amount)

Borrower (Customer name, Loan number)

Depositor (Customer name,Account number)

Find the loan number, branch, amount of loans of greater than or equal to 10000 amount.

$$\{t \mid t \in loan \ \wedge t[amount] >= 10000\}$$

Find the loan number for each loan of an amount greater or equal to 10000.

$$\{t \mid \exists\ s \in loan(t[loan\ number] = s[loan\ number] \wedge s[amount] >= 10000)\}$$

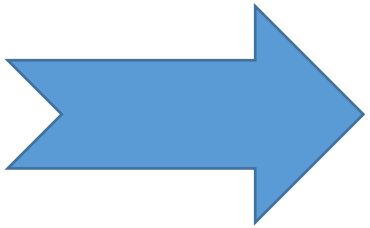
Find the names of all customers who have a loan and an account at the bank.

$$\{t \mid \exists\ s \in borrower(\ t[customer\text{-}name] = s[customer\text{-}name]) \wedge \exists\ u \in depositor(\ t[customer\text{-}name] = u[customer\text{-}name])\}$$

Find the names of all customers having a loan at the "ABC" branch.

$$\{t \mid \exists\ s \in borrower(t[customer\text{-}name] = s[customer\text{-}name] \wedge \exists\ u \in loan(u[branch\text{-}name] = \text{“ABC”} \wedge u[loan\text{-}number] = s[loan\text{-}number]))\}$$

Domain Relational Calculus (DRC)



- Domain Relational Calculus is a non-procedural query language equivalent in power to Tuple Relational Calculus.
- Domain Relational Calculus provides only the description of the query but it does not provide the methods to solve it.
- In tuple relational calculus, the variables range over the tuples whereas in domain relational calculus, the variables range over the domains. The domain variables are the ones which range over the underlying domains instead of over the relations.
- In Domain Relational Calculus, a query is expressed as,

$$\{ < x_1, x_2, x_3, \ldots, x_n > \mid P\ (x_1, x_2, x_3, \ldots, x_n\ ) \}$$

where, $< x_1, x_2, x_3, \ldots, x_n >$ represents resulting domains variables and $P\ (x_1, x_2, x_3, \ldots, x_n\ )$ represents the condition or formula equivalent to the Predicate calculus.

Predicate Calculus Formula:

1. Set of all comparison operators
2. Set of connectives like and, or, not
3. Set of quantifiers

Let Rel $\rightarrow$ be a relation name, X, Y $\rightarrow$ be the domain variables, op $\rightarrow$ an operator in the set {<, $\leq$, >, $\geq$, =, $\neq$}. An Atomic formula in domain relational calculus is one of the following:

- $<x_1, x_2, \ldots x_n> \in$ Rel
- X op Y
- X op Constant or Constant op X

A formula is recursively defined by using the following rules:

- Any atomic formula
- If p and q are formulae, then $\neg$p, p $\wedge$ q, p $\vee$ q, or p $\Rightarrow$ q are also formulae
- If p is a formula that contains X as a domain variable, then $\exists$X(p) and $\forall$ X(p) are also formulae

The quantifiers $\exists$ & $\forall$ are said to bind the domain variable X. Whereas a variable is said to be free in a formula if the formula does not contain an occurrence of a quantifier that binds it

Customer

Customer name	Street	City

Loan

Loan number	Branch name	Amount

Borrower

Customer name	Loan number

Find the loan number, branch, amount of loans of greater than or equal to 100 amount.

$$\{\langle l, b, a\rangle \mid \langle l, b, a\rangle \in loan \wedge (a \geq 100)\}$$

Find the loan number for each loan of an amount greater or equal to 150.

$$\{\langle l\rangle \mid \exists\ b, a\ (\langle l, b, a\rangle \in loan \wedge (a \geq 150)\}$$

Find the names of all customers having a loan at the "Main" branch and find the loan amount.

$$\{\langle c, a\rangle \mid \exists\ l\ (\langle c, l\rangle \in borrower \wedge \exists\ b\ (\langle l, b, a\rangle \in loan \wedge (b = \text{“Main”})))\}$$

Sailor Database

Sailors(sid, sname, rating, age)

Boats(bid, bname, color)

Reserves(sid, bid, day)

Query: Find all sailors with a rating above 7

$\{<I, N, T, A> / <I, N, T, A> \in \text{Sailors} \wedge T > 7\}$

Query: Find the names of sailors who reserved boat 111

$\{<N> / \exists I, T, A\ (<I, N, T, A> \in \text{Sailors} \wedge \exists <Ir, Br, D> \in \text{Reserves}\ (Ir=I \wedge Br=111))\}$

Sailor Database

Sailors(sid, sname, rating, age)

Boats(bid, bname, color)

Reserves(sid, bid, day)

Query: Find the names of sailors who have reserved a green boat

$\{<N>/\exists I, T, A\ (<I, N, T, A> \in \text{Sailors} \wedge <I, Br, D> \in \text{Reserves} \wedge \exists <Br, Bn, \text{'green'}> \in \text{Boats})\}$

Query: Find the names of sailors who have reserved at least 2 boats

$\{<N>/\exists I, T, A\ (<I, N, T, A> \in \text{Sailors} \wedge \exists Br1, Br2, D1, D2(<I, Br1, D1> \in \text{Reserves} \wedge <I, Br2, D2> \in \text{Reserves} \wedge Br1 \neq Br2))\}$

Query: Find the names of sailors who have reserved all boats

$\{<N>/\exists I, T, A(<I, N, T, A> \in \text{Sailors} \wedge \forall <B, Bn, C> \in \text{Boats}\ (\exists <Ir, Br, D> \in \text{Reserves}\ (\ I=Ir \wedge Br=B)))\}$

Query: Find sailors who have reserved all green boats

$\{<I,N,T,A>/<I,N,T,A> \in \text{Sailors} \wedge \forall <B, Bn, C> \in \text{Boats}\ (C=\text{'green'} \Rightarrow \exists <Ir, Br, D> \in \text{Reserves}\ (\ I=Ir \wedge Br=B)))\}$

Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
In TRS, the variables represent the tuples from specified relation.	In DRS, the variables represent the value drawn from specified domain.
A tuple is a single element of relation. In database term, it is a row.	A domain is equivalent to column data type and any constraints on value of data.
In this filtering variable uses tuple of relation.	In this filtering is done based on the domain of attributes.
Query cannot be expressed using a membership condition.	Query can be expressed using a membership condition.
The QUEL or Query Language is a query language related to it,	The QBE or Query-By-Example is query language related to it.
It reflects traditional pre-relational file structures.	It is more similar to logic as a modelling language.
Notation : {T | P (T)} or {T | Condition (T)}	Notation : { a1, a2, a3, ..., an | P (a1, a2, a3, ..., an)}
Example : {T | EMPLOYEE (T) AND T.DEPT_ID = 10}	Example : { | < EMPLOYEE > DEPT_ID = 10 }


The End